

Създаване на програми с класове и обекти. Наследяване. Обработка на грешки.

I. Теоретична постановка

1. Наследяване

В ООП може да се създава йерархия от класове. За всеки клас може да се обяви клас наследник посредством следния синтаксис:

```
class <Име на клас наследник>(<Име на родителски клас>):  
    <описание на атрибутите и методите на класа наследник>
```

В класа наследник ще бъдат достъпни всички атрибути и методи на родителския (базовия) клас. Те се допълват с нови методи и атрибути. Някои от методите на базовия клас могат да се преопределят (да се променят параметрите и/или инструкциите в тях без да се сменя името на метода) в класа наследник. В този случай за да извикаме копието на метода от родителския клас, може да се използва функция `super()`. Тя се използва без аргументи.

2. Обработка на грешки

Изключението е съобщение на интерпретатора за грешка в програмния код или друго събитие. Ако в кода на програмата не се обработват тези изключения, изпълнението на програмата се прекратява и се извежда съобщение за грешка. Голяма част от изключенията възникват по време на изпълнението на програмата поради грешки, които не са била предвидена от програмиста. Често срещани типове изключения са:

`IndexError` – генерира се ако в последователност не бъде намерен елемент с посочения индекс

`KeyError` - генерира се ако в речник не бъде намерен указания ключ

`NameError` - генерира се ако не бъде намерено име (на променлива или функция)

`TypeError` - генерира се ако стандартна операция се прилага към обект от неподходящ тип

`ValueError` - генерира се ако операция или функция приемат аргумент с неподходяща стойност

`ZeroDivisionError` - генерира се при деление на 0

Обработката на изключения може да се направи с инструкция `try..except`. Използва се следния формат:

`try:`

 <Код, в който може да възникнат изключения>

`except <Изключение1>:`

 <Код, който се изпълнява при възникване на Изключение1>

....

`except <ИзключениеN>:`

 <Код, който се изпълнява при възникване на ИзключениеN>

`else:`

 <Код, който се изпълнява ако не възникне изключение>

`finally:`

<Код, който ще бъде изпълнен при всеки един случай>

Частта else и finally се използват по-рядко. Основно се прилагат блокове try и except.

Пример 1: Да се напише програма, която създава клас Parent с атрибут едно цяло число. В класа да се включат метод за извеждане на стойността на числото. Да се създаде клас Child (наследник на клас Parent) с атрибути две цели числа. Да се създадат един обект от родителския клас и един обект от класа наследник и за тях да се извика метода за извеждане на обекта. При въвеждане на данните за обектите да се контролира дали са въведени целочислени стойности от клавиатурата.

```
class Parent:
    # Родителски клас
    """Родителски клас"""
    def __init__(self,a):
        # Конструктор на класа
        self.a=a
    def show(self):
        # Метод за извеждане на данните за обекта
        print(f"a = {self.a}")
class Child(Parent):
    # Клас наследник
    """Клас наследник"""
    def __init__(self,a,b):
        # Конструктор на класа
        super().__init__(a)
        self.b=b
    def show(self):
        # Метод за извеждане на данните за обекта
        print(f"a = {self.a}  b = {self.b}")

def control_int():
    # Функция за контрол на входните данни
    """Контрол на входните данни"""
    while True:
        # Обработка на изключение
        try:
            num=int(input("Въведете цяло число >> "))
        except ValueError:
            print(control_int.__doc__)
            print("Необходимо е да се въведе цяло число!")
        else:
            print("Не възникна изключение!")
            return num
    finally:
        print("Входните данни бяха обработени!")

print("Създаване на обект от клас Parent: ")
x=control_int()
P=Parent(x)
# Създаване на обект от родителския клас
print("Отпечатване на обект от клас Parent: ")
P.show()
# Викане на метод show()
print("Създаване на обект от клас Child: ")
x=control_int()
```

```
y=control_int()  
Ch=Child(x,y)           # Създаване на обект от клас наследник  
print("Отпечатване на обект от клас Child: ")  
Ch.show()               # Викане на метод show()
```

Пример 2: Да се напише програма на Python, която създава клас студент с атрибути име и възраст (цяло число) и име на университет - ТУ. В класа да се включат метод за извеждане на данните за студента. Да се създаде клас наследник факултет с атрибути име на факултет и факултетен номер. Да се въведе информацията за избран брой студенти (цяло число). Да се изведат основните данни (име и възраст) за всички студенти на ТУ. Да се изведат всички записани данни за студентите само от избран факултет. Да се правят проверки за възникване на изключителни ситуации при въвеждане на данните за целочислените аргументи.

```
class Student:           # Клас студент  
    """Клас за студенти в ТУ"""  
    university="Технически университет - София"  
    def __init__(self,name,age):  
        self.name=name  
        self.age=age  
    def show_st(self):  
        print(f"{self.name} {self.age}")  
class Faculty(Student):  # Клас факултет  
    """Клас за студенти от факултет"""  
    def __init__(self,name,age,fk_name,number):  
        super().__init__(name,age)  
        self.fk_name=fk_name  
        self.number=number  
    def show_fk(self):  
        print(f"{self.name} {self.fk_name} {self.number}")  
    def get_fk_name(self):  
        return self.fk_name  
def control_int():       # Функция за контрол на входните данни  
    """Контрол на входните данни"""  
    while True:  
        try:             # Обработка на изключение  
            num=int(input("Въведете цяло число >> "))  
            return num  
        except ValueError:  
            print(control_int.__doc__)  
            print("Необходимо е да се въведе цяло число!")  
  
def add_student():       # Функция за въвеждане на данни за студент  
    """Добавяне на данни за студент"""  
    x=input("Име: ")  
    print("Възраст: ")
```

```
y=control_int()
z=input("Факултет: ")
n=input("Факултетен номер: ")
F=Faculty(x,y,z,n)      # Създаване на обект от клас факултет
return F
def input_student(st_list):  # Функция за въвеждане на данни за група студенти
    """Въвеждане за група студенти"""
    print("Колко студента ще въведете?")
    br=control_int()
    for i in range(0,br):
        F=add_student()
        st_list.append(F)
    return st_list
def show_student(st_list):  # Функция за извеждане на основни данни всички студенти
    """Извеждане на списък със студенти на ТУ"""
    if st_list!=[]:
        print(Student.university)
        for i in range(0,len(st_list)):
            st_list[i].show_st()
    else:
        print("Списъкът е празен!")
def show_fakulty(st_list,fk): # Функция за извеждане на студенти от избран факултет
    """Извеждане на списък със студенти във факултет"""
    for i in range(0,len(st_list)):
        if st_list!=[]:
            fname=st_list[i].get_fk_name()
            if fname==fk:
                st_list[i].show_fk()
        else:
            print("Списъкът е празен!")

lst=[]      # Създаване на празен списък
print(input_student.__doc__)
lst=input_student(lst)
print(show_student.__doc__)
show_student(lst)
key=input("Въведете име на факултет: ")
print(show_fakulty.__doc__)
show_fakulty(lst,key)
```

II. Задачи за самостоятелна работа

1. Направете промени в кода на Пример 1, така че в двата класа да се добави метод за повдигане на квадрат на аргументите на обектите. В програмата да се викат новите методи за обектите и на двата класа.

Забележка: Повдигане на квадрат на дадено число може да направи с функция `math.pow()` или като стойността на числото се умножи по себе си.

Помислете как в метод `show()` на класа наследник може да се извика методът със същото име от класа родите.

2. Направете промени в кода на Пример 2, така че в клас факултет да се добави като аргумент и номер на курс (цяло число) на студента. Да могат да се извеждат данните на студентите от избран курс. Да се контролират входните данни за аргумента курс на студента. Те трябва да са цяло число в интервала от 1 до 4 включително.

Помислете как да изведете данните на студенти от избран факултет, които са и от избран курс.